Introduction to Tree Data Structures

Introduction to Trees

This section introduces the motivation and concepts behind general tree data structures.

Content

- Motivation
- Terminology
- Implementation
- Tree Traversal

Motivation

Inserting or deleting can be very efficient for linked lists, but finding values in a linked list is not efficient.

Conversely, finding values in a sorted array can be very efficient, but inserting into a sorted array is not.

Function	List	Array
Insert at Front	$O(1)$	$O(n)$
Insert in Middle	$O(n)$	$O(n)$
Insert at End	$O(n)$ $O(1)$ (with tail pointer)	$O(n)$ $O(1)$ (with count)
Find	$O(n)$	$O(n)$ $O(\log n)$ (if sorted)

It's clear that if we want to improve on $O(n)$ for all operations, we are going to need a data structure with more, well, structure. Trees give us more structure, so let's see if that helps.

Tree Terminology

Let's start with some tree terminology.

A *graph* is a set of nodes with links or *edges* between them.

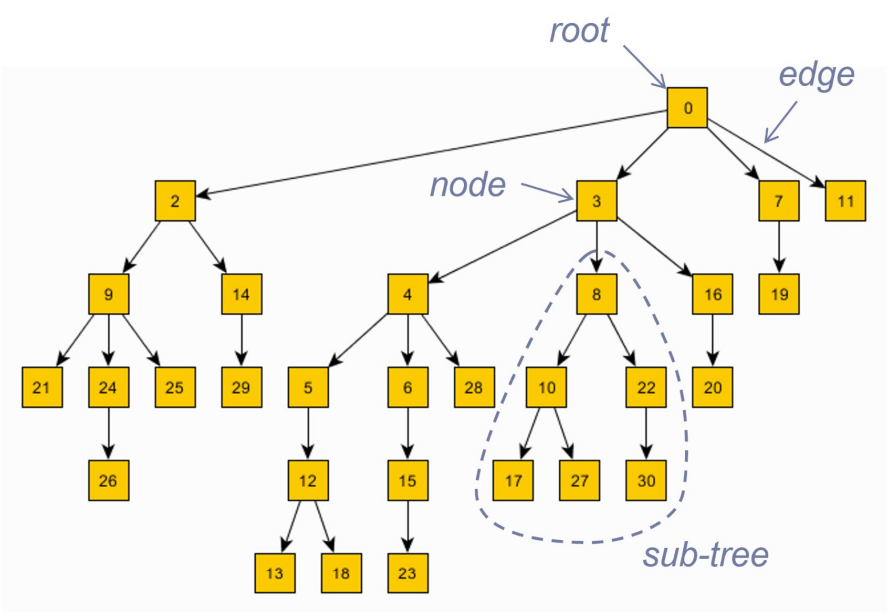
A tree is a special kind of graph.

We can degine a tree recursively.

A tree is either:

- empty, or
- consists of a *root* node, and zero or more (sub)trees connected to the root

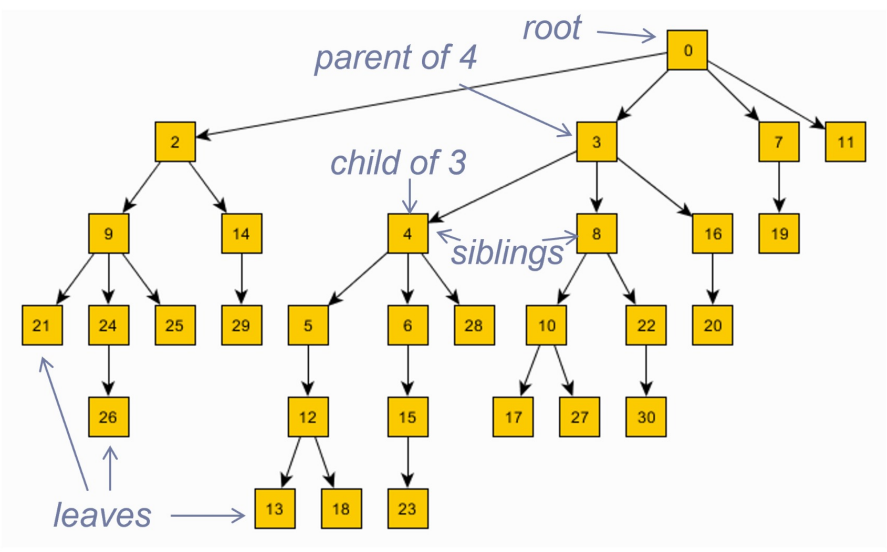
Note that we usually draw trees with the root at the top and work down.



If two nodes are connected by an *edge* we say that the nodes are *adjacent*.

Some family tree terminology carries over to the computing area:

- A node that is adjacent to a node and above it, is its *parent*.
- One that is adjacent and below it is its *child*.
- Such terms as *siblings* and *ancestors* can also be used.



Some wood style tree terminology also carries over to tree data structures:

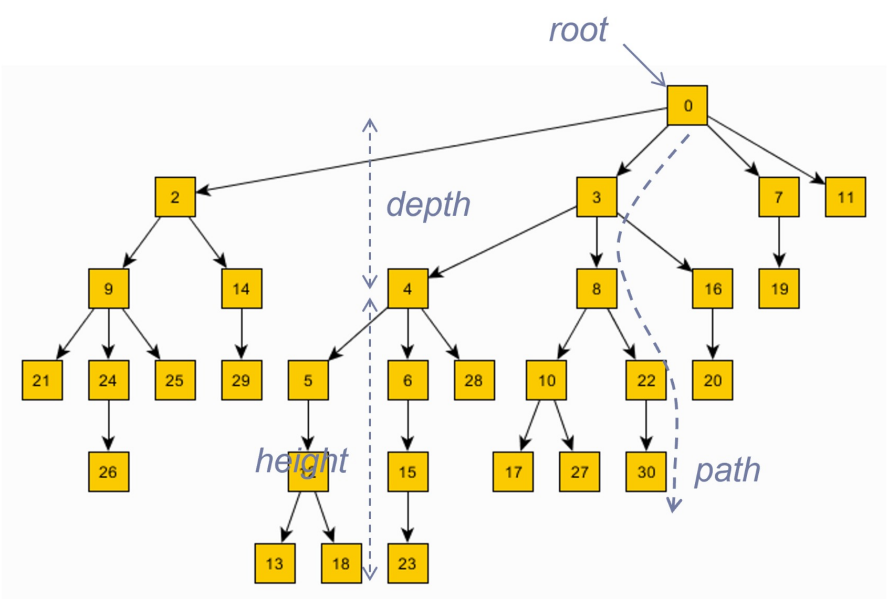
- A node with no children is a *leaf*. (A node that is not a leaf is called an *internal node*)
- A subtree can also be called a *branch*.
- A collection of trees is a *forest*.

A *path* is a sequence of nodes, each the parent of the next in the path.

The length of the path (number of edges) to a node from the root is the node's *depth*. In the diagram, the depth of node '4' is 2; the depth of node '18' is 5.

The *height* of a node is the length of the longest path from it to a leaf. In the diagram, the height of node '4' is 3; the height of node '7' is 1.

The height of a tree is the height of its root. The height of the tree in the diagram is 5.

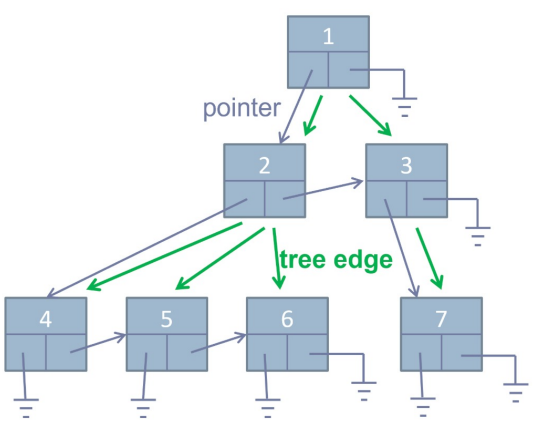


Tree Implementation

General trees can be implemented in different forms of linked data structures. A common technique is to use a "first child, next sibling" representation. The following data structure can be used.

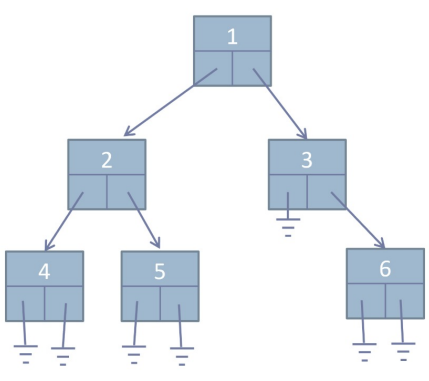
```
struct treeNode{
    int data;
    struct treeNode *first_child;
    struct treeNode *next_sibling;
};
```

Using this data structure, the blue pointers in the diagram would be used to represent the green tree edges.



Some trees are more constrained and can be implemented in a way to capitalise on this structure. For example, binary trees (a tree where each node has at most two children) can be represented with the following code:

```
struct binaryTreeNode{
    int data;
    struct binaryTreeNode *left;
    struct binaryTreeNode *right;
};
```



Tree Traversal

Traversal is the process of visiting every item in a collection. For singly linked lists, there is only one way to traverse the data structure - start at the head and move to the next node at each step of the traversal.

But for trees, there is more than one way to traverse. If we visit children in the order left-to-right (or in sibling order), then we have two traversal methods for general trees:

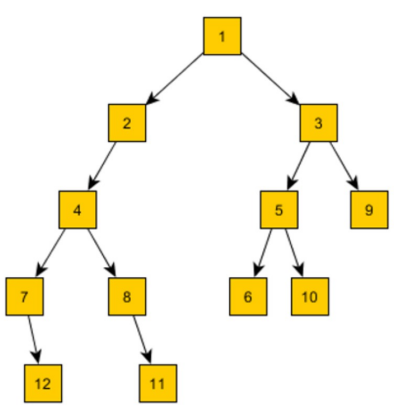
- **Pre-order:** A node is considered before its children.
- **Post-order:** A node is considered after its children.

For binary trees, we also have a third method:

- **In-order:** A node is considered after its left child and before its right child.

So, for the tree shown, these traversals would visit nodes in the following order:

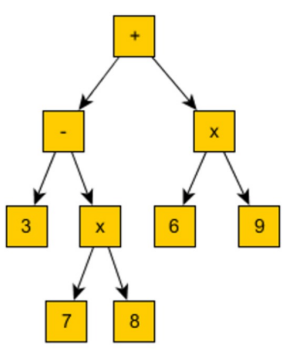
- **Pre-order:** 1, 2, 4, 7, 12, 8, 11, 3, 5, 6, 10, 9
- **In-order:** 7, 12, 4, 8, 11, 2, 1, 6, 5, 10, 3, 9
- **Post-order:** 12, 7, 11, 8, 4, 2, 6, 10, 5, 9, 3, 1



A particular type of binary tree is the *expression tree*. In an expression tree, internal nodes are binary mathematical operators and the leaves are numbers.

Pre-order, in-order, and post-order traversals of an expression tree yield the prefix, infix and postfix forms of the expression represented. For example, the following expressions result from the tree shown:

- **Prefix:** + - 3 x 7 8 x 6 9
- **Infix:** 3 - 7 x 8 + 6 x 9
- **Postfix:** 3 7 8 x - 6 9 x +



Some old Hewlitt Packard calculators famously used postfix notation (referred to as Reverse Polish Notation for historical reasons) since you never need to use parentheses (parenthesis may need to be added to infix expressions to override order of operations).

Summary

Any data which involves an inherent hierarchy is can easily be represented using a tree data structure. Hierarchical structures are used in the real world for efficient organisation. So maybe they will also improve efficiency for storing and retrieving data.

We discussed tree terminology and implementation, and also looked at different types of trees and different ways to traverse trees.

Next we will look at search trees.